Status: PREPARED DRAFT

Session 15 — Study Guide: From Issuing a JWT to Trusting a Request

Where the session ends

A real, signed-in user can be verified on any request. `GET /auth/me` proves identity end to end. Live code builds this much. Teamwork then applies it, for the first time, to every write route across all three real resources — Issues, Projects, AND Workspaces, in one block, not spread across different parts of the session. `InternalToolGuard` still correctly protects `GET /projects/stats` — the one deliberate exception. No role, no ownership check, no enforcement exists anywhere yet during live code or teamwork — any signed-in user may write any resource. Homework then begins building the real DATA Session 16 will need to eventually answer that question — without answering it itself (see "A preview of Session 16" below).

The real starting point

Session 14's real classroom delivery reached registration only. The rest of Session 14's own intended scope (login, `bcrypt.compare()`, JWT issuance, email normalization) was completed as the Session 14 Completion Homework. This session begins from that homework's own finished state — confirm your own repository is actually caught up before class, and ask for a short individual recovery if it isn't.

The problem: issuing a token is not the same as trusting one

Login already returns a real, signed JWT:

JSON

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIs...",
  "user": { "id": 1, "name": "...", "email": "...", "createdAt": "..." }
}
```

Nothing on this API currently reads, checks, or trusts that token on any LATER request. Every write route still either checks the old shared internal-tool key, or nothing at all. Today builds the missing half: a request arrives carrying a token — now what?

Decode vs. verify

A JWT's own payload is always readable by anyone holding the token — it is signed, not encrypted. Reading it back out (decoding) proves nothing: a tampered, forged, or entirely made-up token can still be decoded successfully. Verifying recomputes the token's own signature using this server's own secret key and rejects the token if it doesn't match — that is the one operation that actually proves this server issued it and nobody altered it since.

TEXT

```
jwtService.decode(token)        -> proves nothing about authenticity
jwtService.verifyAsync(token)   -> actually verifies the signature (and checks expiry
automatically)
```

`JwtAuthGuard` — the same shape as a guard you already know

TS

```ts
@Injectable()
export class JwtAuthGuard implements CanActivate {
  constructor(private readonly jwtService: JwtService) {}

  async canActivate(context: ExecutionContext): Promise<boolean> {
    const request = context.switchToHttp().getRequest<Request>();
    const token = this.extractToken(request);

    if (!token) {
      throw new UnauthorizedException(
        'Invalid or missing authentication token.',
      );
    }

    try {
      request.user = await this.jwtService.verifyAsync(token);
    } catch {
      throw new UnauthorizedException(
        'Invalid or missing authentication token.',
      );
    }

    return true;
  }
}
```

Same `CanActivate` interface `InternalToolGuard` already uses — a different question. A missing header, a wrong scheme, a malformed token, a bad signature, and an expired token are all rejected with the IDENTICAL external `401` — same message, same shape — the caller never learns which case it was, the same anti-enumeration reasoning already used for login failures in Session 14: a distinct message per case would let someone holding a stolen or guessed token learn, one bit at a time, exactly why it failed.

`@CurrentUser()` — reading what the guard already proved

TS

```
export const CurrentUser = createParamDecorator(
  (_data: unknown, context: ExecutionContext) => {
    const request = context.switchToHttp().getRequest<Request>();
    return request.user;
  },
);
```

This only works on a route that already has `@UseGuards(JwtAuthGuard)` — without it, `request.user` is simply `undefined`. The decorator does no verification itself; it only reads what the guard already set.

`GET /auth/me` — why it does its own lookup

TS

```
@UseGuards(JwtAuthGuard)
@Get('me')
async me(@CurrentUser() claims: JwtPayload) {
  const user = await this.usersService.findById(claims.sub);
  if (!user) {
    throw new UnauthorizedException('This account no longer exists.');
  }
  return this.usersService.toSafeUser(user);
}
```

`claims` only has `{ sub, email }` — exactly what was true the moment the token was signed, up to 8 hours ago. A signature proves the claims weren't TAMPERED WITH; it says nothing about whether they're still CURRENT. `/auth/me` is specifically the "prove who I am, right now" endpoint, so it performs its own fresh database read instead of trusting the token's claims directly.

Protecting a real write route

TS

```
@ApiBearerAuth()
@UseGuards(JwtAuthGuard)
@Post()
create(@Body() dto: CreateIssueDto) {
  return this.issuesService.create(dto);
}
```

`InternalToolGuard` answered "does the caller hold our shared internal-tool key" — true or false of a TOOL, not a person. `JwtAuthGuard` answers "is this a real, currently signed-in user." Only three routes change per resource (create/update/delete); read routes stay open. `GET /projects/stats` deliberately keeps `InternalToolGuard` — it's called by this app's own internal tooling, not by a human with a registered account.

Not every route needs the same guard — Swagger needs to know about both

TS

```
.addBearerAuth()
```

That one line, at the document level, registers the scheme Swagger UI's own Authorize button offers. Then, per route:

TS

```
@ApiBearerAuth()
```

This says THIS specific route needs a Bearer token. Both are required — `@ApiBearerAuth()` alone documents the requirement but gives Swagger UI nothing to attach a token to; `.addBearerAuth()` alone registers a scheme nothing uses yet.

Common beginner mistakes

- Treating "the token exists" as the same thing as "the API is protected" — Session 14 explicitly stopped at issuing one; nothing verified one until today.
- Using `decode()` where `verifyAsync()` belongs.
- Assuming a JWT is encrypted — its payload is always readable by whoever holds it; nothing secret belongs there.
- Registering a guard globally instead of per-route — a global JWT guard would make `POST /auth/login` itself require a token, which is impossible.
- Returning `request.user`'s claims directly from `/auth/me` instead of doing a fresh lookup.

A preview of Session 16 — authentication is not authorization

Today answers WHO is making a request. It never asks WHAT they're allowed to do with it — any signed-in user may currently write any resource, on purpose. That is a real, separate question: knowing someone is genuinely logged in says nothing about whether THIS workspace should let THEM create a project in it, or delete someone else's issue. Answering that eventually needs a new fact this app doesn't track yet: which workspaces a user belongs to, and what role they hold in each one — the same person might be the Owner of one workspace and an ordinary Member of another. Your own homework begins building exactly that real data (a `WorkspaceMember` recording a user's role in a specific workspace) — but builds no guard, no permission check, and no enforcement with it. Session 16 is where that data actually starts deciding anything.

Not yet

No roles (Owner/Admin/Member) enforced anywhere, no ownership-based checks (assignee, reporter), no guard or route that reads a role to allow or deny a request. Any signed-in user may currently write any resource — that enforcement boundary is next session's own material.

Glossary

- **Claim** — a key/value pair inside a JWT's payload (`sub`, `email`, plus `iat` / `exp`).
- **Bearer token** — a token that grants access to whoever holds it, no further proof required.
- **Guard** — a Nest `CanActivate` class deciding whether a request may proceed.
- **Signing key** — the secret used to compute and verify a JWT's signature.

Review questions

1. What's the actual difference between decoding a JWT and verifying one?
2. Why does `GET /auth/me` do its own database lookup instead of returning `request.user` directly?
3. Why does `GET /projects/stats` keep using `InternalToolGuard` instead of switching to `JwtAuthGuard` like every other write route?
4. What would break if `JwtAuthGuard` were registered globally instead of per-route?

Answers

1. Decoding reads the payload back out with no cryptographic check at all — it can't tell a real token from a forged one. Verifying recomputes the signature with this server's own key and rejects anything that doesn't match (plus checks expiry).
2. A token's claims only reflect the truth at the moment it was issued; they could be stale by the time a later request arrives (a changed email, a deleted account). `/auth/me` is specifically the identity-proof endpoint, so it checks current truth, not token-time truth.
3. `GET /projects/stats` is called by this app's own internal tooling, not a human with an account — the shared-secret question is the right one for that caller; a real signed-in-user check doesn't apply.
4. Every route, including `POST /auth/login` and `POST /auth/register`, would then also require a valid token — making it impossible to ever log in and obtain one in the first place.

What you should be able to explain now

- The difference between decoding and verifying a JWT, and why only one of them is trustworthy.
- How a custom Nest guard attaches identity to a request, and how a custom decorator then exposes it to a controller.
- Why identity verification (authentication) and permission checking (authorization) are different questions, and which one this session answers.
- Why `/auth/me` performs a fresh lookup instead of trusting token claims directly.